

Functional Programming using map:

1.Functional Programming using map:

```
class Solution {  
    public int[] buildArray(int[] nums) {  
        int[] ans = new int[nums.length];  
        for (int i = 0; i < nums.length; i++) {  
            ans[i] = nums[nums[i]];  
        }  
        return ans;  
    }  
}
```



2. Shuffle the Array

```
class Solution {  
    public int[] shuffle(int[] nums, int n) {  
        int[] result = new int[2 * n];  
        for (int i = 0; i < n; i++) {  
            result[2 * i] = nums[i];  
            result[2 * i + 1] = nums[i + n];  
        }  
        return result;  
    }  
}
```



Maximum XOR of Two Numbers in an Array

Description: Given an array of numbers, return the maximum value of $a \oplus b$ where a and b are two numbers from the array.

Constraints:

- $1 \leq \text{nums.length} \leq 10^4$
- $0 \leq \text{nums[i]} < 2^{31}$

Example 1:

```
Input: nums = [3, 10, 5, 25, 2, 8]
Output: 35
Explanation: The maximum value of a XOR b is 35 (5 XOR 30).
```

Example 2:

```
Input: nums = [14, 40, 39]
Output: 57
Explanation: (14 XOR 40) = 54, (40 XOR 39) = 1, (14 XOR 39) = 57.
```

Accepted: 1,206,425 / 1,206,425
Acceptance Rate: 88.7%

Tags: Bit Manipulation, Trie

Complexity: Time: $O(N \cdot 32)$, Space: $O(32 \cdot 2^{31})$

Solution:

```
class TrieNode {
public:
    bool isLeaf;
    TrieNode* child[2];
    TrieNode() {
        isLeaf = false;
        child[0] = child[1] = nullptr;
    }
};

class Trie {
public:
    TrieNode* root;
    Trie() {
        root = new TrieNode();
    }
    void insert(int num) {
        TrieNode* node = root;
        for (int i = 31; i >= 0; i--) {
            int bit = (num >> i) & 1;
            if (node->child[bit] == nullptr) {
                node->child[bit] = new TrieNode();
            }
            node = node->child[bit];
        }
        node->isLeaf = true;
    }
    int getMaxXOR(int num) {
        TrieNode* node = root;
        for (int i = 31; i >= 0; i--) {
            int bit = (num >> i) & 1;
            int oppBit = 1 - bit;
            if (node->child[oppBit] != nullptr) {
                node = node->child[oppBit];
            } else {
                node = node->child[bit];
            }
        }
        return num ^ (node->isLeaf ? num : 0);
    }
};

int findMaximumXOR(vector<int>& nums) {
    Trie trie;
    for (int num : nums) {
        trie.insert(num);
    }
    int max = 0;
    for (int num : nums) {
        max = max > trie.getMaxXOR(num) ? max : trie.getMaxXOR(num);
    }
    return max;
}
```